Complete Project Documentation

AI-Powered Intelligent Document Management and Question-Answering System built on Retrieval-Augmented Generation (RAG)

Project Name	DocuMind AI
Project Type	Retrieval-Augmented Generation (RAG) application, full stack
Frontend	React 19, Vite 8 (deployed on Vercel)
Backend	Python, FastAPI (Docker container on Render)
Database	MongoDB (MongoDB Atlas in production)
AI Integration	Google Gemini (embeddings and answer generation)
GitHub Repository	https://github.com/raja-oss-official/documind-ai
Live Deployment	https://documind-ai-wine.vercel.app
API Documentation	https://documind-ai-api-83az.onrender.com/docs
Developer	Raja Sundaram R S · Hexaware Internship Project
Document Date	October 29, 2026

Table of Contents

1. Abstract . . . 5

2. Introduction . . . 5

3. Problem Statement . . . 6

4. Proposed Solution . . . 6

5. Project Objectives . . . 7

6. Scope of the Project . . . 7

7. System Architecture . . . 8

7.1 Layers and responsibilities . . . 8

7.2 Request lifecycle . . . 9

8. Technology Stack . . . 9

9. Project Structure . . . 10

9.1 Backend service modules . . . 10

9.2 Frontend components . . . 11

10. Database Design . . . 12

10.1 Collections and fields . . . 12

10.2 Indexes . . . 12

11. Authentication and Security . . . 13

11.1 Password security . . . 13

11.2 Access and refresh tokens . . . 13

11.3 Authorisation and data isolation . . . 14

11.4 Rate limiting . . . 14

11.5 Security headers . . . 14

11.6 Upload security . . . 14

11.7 Error handling, CORS and secrets . . . 15

12. Document Processing Pipeline . . . 15

12.1 Text extraction by file type . . . 15

12.2 Sentence-aware chunking . . . 15

12.3 Embeddings . . . 16

12.4 Status tracking . . . 16

13. Hybrid Semantic Search . . . 16

13.1 Scoring formula . . . 16
13.2 Search parameters . . . 16
13.3 Contextual neighbour boosting . . . 17
13.4 Local similarity and optional vector search . . . 17
13.5 Search response . . . 17

14. Retrieval-Augmented Generation (RAG) . . . 17

14.1 Step 1: Retrieval . . . 18
14.2 Step 2: Choosing the context strategy . . . 18
14.3 Step 3: Prompt construction . . . 18
14.4 Step 4: Generation with model fallback . . . 19
14.5 Step 5: Response and sources . . . 19

15. Document Summaries . . . 19

15.1 Direct summary (active) . . . 19
15.2 Map-reduce summary (implemented, not yet connected) . . . 20

16. AI Chat and Document Intelligence . . . 20

16.1 Backend behaviour . . . 20
16.2 Frontend experience . . . 20
16.3 Example questions . . . 20

17. API Reference . . . 21

17.1 Health . . . 21
17.2 Authentication . . . 21
17.3 Documents . . . 21
17.4 Chats . . . 22
17.5 Example: login and ask . . . 22

18. Frontend Application . . . 22

18.1 Pages . . . 22
18.2 API client (api.js) . . . 23
18.3 Design and responsiveness . . . 23

19. Configuration . . . 24

19.1 Backend environment variables (backend/.env) . . . 24
19.2 Frontend environment variable . . . 24
19.3 Tuning constants in code . . . 24

20. Deployment . . . 25

20.1 Backend container (Dockerfile) . . . 25
20.2 Render blueprint (render.yaml) . . . 25

21. Automated Testing 26
21.1 Running the tests 26
22. Performance and Optimisation 27
23. Version Control and GitHub 27
23.1 Commit history 27
23.2 Ignored files (.gitignore) 27
24. Local Setup Guide 28
25. Current Project Status 28
25.1 Testing preparation 29
26. Limitations 29
27. Future Enhancements 30
28. Conclusion 30
29. Appendix: Glossary 31

1. Abstract

DocuMind AI is an AI-powered document intelligence workspace. A user creates an account, uploads documents (PDF, Word, Excel, PowerPoint, CSV, plain text, or images), and can then **search them by meaning**, **ask questions** in a ChatGPT-style chat, and **generate summaries with key points**. All AI answers are grounded only in the user's own documents and are returned together with the source passages they were drawn from.

Traditional document management systems mainly offer storage and exact keyword search. People still have to open long files and read them manually to find a definition, a figure, a deadline or a conclusion. DocuMind AI removes most of that effort by combining **text extraction**, **Optical Character Recognition (OCR)**, **sentence-aware chunking**, **vector embeddings**, **hybrid semantic search** and **Retrieval-Augmented Generation (RAG)** with Google Gemini.

The system is a complete full-stack application. The backend is a FastAPI service (about 8,000 lines of Python across routes, schemas, core modules and 15 services) backed by MongoDB. The frontend is a responsive React 19 + Vite 8 single-page application (about 8,100 lines of JSX and CSS). Security features include Argon2 password hashing, short-lived JWT access tokens, rotating hashed refresh tokens, rate limiting, strict security headers, upload signature validation and per-user data isolation on every query.

The project is version-controlled on GitHub and deployed publicly: the frontend on **Vercel**, the backend as a **Docker container on Render**, and the database on **MongoDB Atlas**. A suite of **63 automated pytest tests** covers authentication, password security, refresh-token rotation, error handling, database indexes and query plans, chunking and search performance, and an end-to-end backend flow.

2. Introduction

The volume of digital information keeps growing in education, research, business, administration and personal work. Documents hold valuable knowledge, but locating one specific detail inside a 60-page report, a scanned handout or a spreadsheet export is slow and frustrating.

Consider a student reading a research paper who wants its objectives, methodology, results and conclusion; a professional who must find one clause in a long policy; or an intern who needs the action items from a slide deck. Reading every page is inefficient, and keyword search often fails because the question uses different words from the document (for example, asking about *"cost"* when the document says *"expenditure"*).

General-purpose AI chatbots can answer questions fluently, but they answer from their training data, not from the user's files, and they can invent facts ("hallucinate"). DocuMind AI addresses both problems. It first **retrieves** the most relevant passages from the user's own documents and then asks the language model to **generate** an answer using only those passages. This technique is called Retrieval-Augmented Generation.

The project combines conventional backend engineering (REST APIs, authentication, database design, testing, deployment) with modern AI techniques (embeddings, semantic search, prompt engineering and grounded generation) to deliver a practical, secure document assistant.

3. Problem Statement

Users who work with digital documents face the following problems:

- **Time cost:** finding specific information in long documents takes considerable time.
- **Vocabulary mismatch:** keyword search fails when the question uses different words from the document.
- **Scanned content:** scanned PDFs and photos contain images of text that cannot be searched directly.
- **Comprehension:** large research papers, reports and manuals are hard to understand quickly.
- **Fragmented tools:** switching between a document reader, a search tool and an AI assistant interrupts the workflow.
- **Untrustworthy AI answers:** generic chatbots may answer from outside knowledge or invent facts, with no way to verify the source.
- **Privacy and isolation:** documents belonging to different users must be strictly separated.
- **Reliability:** the application needs robust authentication, error handling and efficient database operations to protect user data.

There is therefore a need for one integrated system that combines document processing, meaning-based retrieval, grounded AI answers with sources, and secure user-specific access.

4. Proposed Solution

DocuMind AI is an integrated platform for document storage, processing, search, summarisation and AI-assisted question answering. The solution works in four stages:

1. **Ingest.** When a file is uploaded, the backend validates it, extracts its text page by page (using OCR for scanned PDFs and images), splits the text into overlapping 500-character chunks, and converts each chunk into a numeric vector (embedding) with Google's *gemini-embedding-2* model.
2. **Retrieve.** When the user searches or asks a question, the question is embedded in the same vector space and compared with every chunk the user owns. A hybrid score combines semantic similarity (75%) with keyword overlap (25%), and passages next to strong matches receive a small contextual boost.
3. **Generate.** The best passages (or the whole document, for broad questions and small documents) are placed in a carefully designed prompt with strict grounding rules and recent conversation history. Gemini writes a structured Markdown answer.
4. **Present.** The React frontend shows the answer with headings, lists, tables and code blocks, together with the source passages, document names, page numbers and match scores.

The application is built with a modular architecture so that authentication, document processing, AI integration, search and chat can be developed, tested and maintained independently.

5. Project Objectives

#	Objective	Status
1	Build a document management application with an AI interaction layer	Achieved
2	Provide secure registration, login, session refresh and logout	Achieved
3	Restrict every document, chunk, chat and search to its owner	Achieved
4	Extract text from PDF, DOCX, TXT, CSV, XLSX, PPTX, PNG and JPG files	Achieved
5	Use OCR for scanned PDFs and images	Achieved (Tesseract, English)
6	Split long documents into page-aware chunks	Achieved
7	Generate embeddings and implement meaning-based (hybrid) search	Achieved
8	Answer questions with RAG, grounded in user documents, with sources	Achieved
9	Provide conversational chat with history and document scoping	Achieved
10	Generate document summaries and key points	Achieved (direct summary)
11	Build a responsive React interface connected to the API	Achieved
12	Maintain automated tests for important behaviour	Achieved (63 tests)
13	Use GitHub for version control and deploy the app online	Achieved

6. Scope of the Project

The project focuses on **intelligent interaction with documents**, not just file storage. The implemented scope is:

Area	Included capabilities
User management	Registration with strong-password rules; login; JWT access tokens; rotating refresh tokens; logout; change password (signs out all devices); current-user profile
Document management	Upload by button or drag and drop (max 10 MB); validation; text extraction; OCR; chunking; embeddings; statistics (pages, words, characters, chunks); rename; delete; status tracking
Intelligent search	Hybrid semantic + keyword search; neighbour boosting; document filter; 5/10/20 result selector; score breakdown; optional MongoDB Atlas vector search
AI assistance	Ask AI across all documents; chat sessions linked to one document or all documents; conversation memory; whole-document mode for broad questions; summaries and key points
Frontend	Sign-in/register pages; dashboard; documents library; document details; AI Search; AI Chat; Account page; desktop, tablet and phone layouts
Operations	Docker image with OCR dependencies; Render blueprint; Vercel deployment; environment-based configuration; MongoDB Atlas

Out of scope for the current version: team workspaces and document sharing, true token-by-token streaming, background job queues, multilingual OCR, and document editing.

7. System Architecture

DocuMind AI follows a layered **client-server architecture**. The browser runs a React single-page application that talks to a stateless FastAPI REST API over HTTPS. The API delegates work to focused service modules, which use MongoDB for persistence and the Google Gemini API for embeddings and text generation.

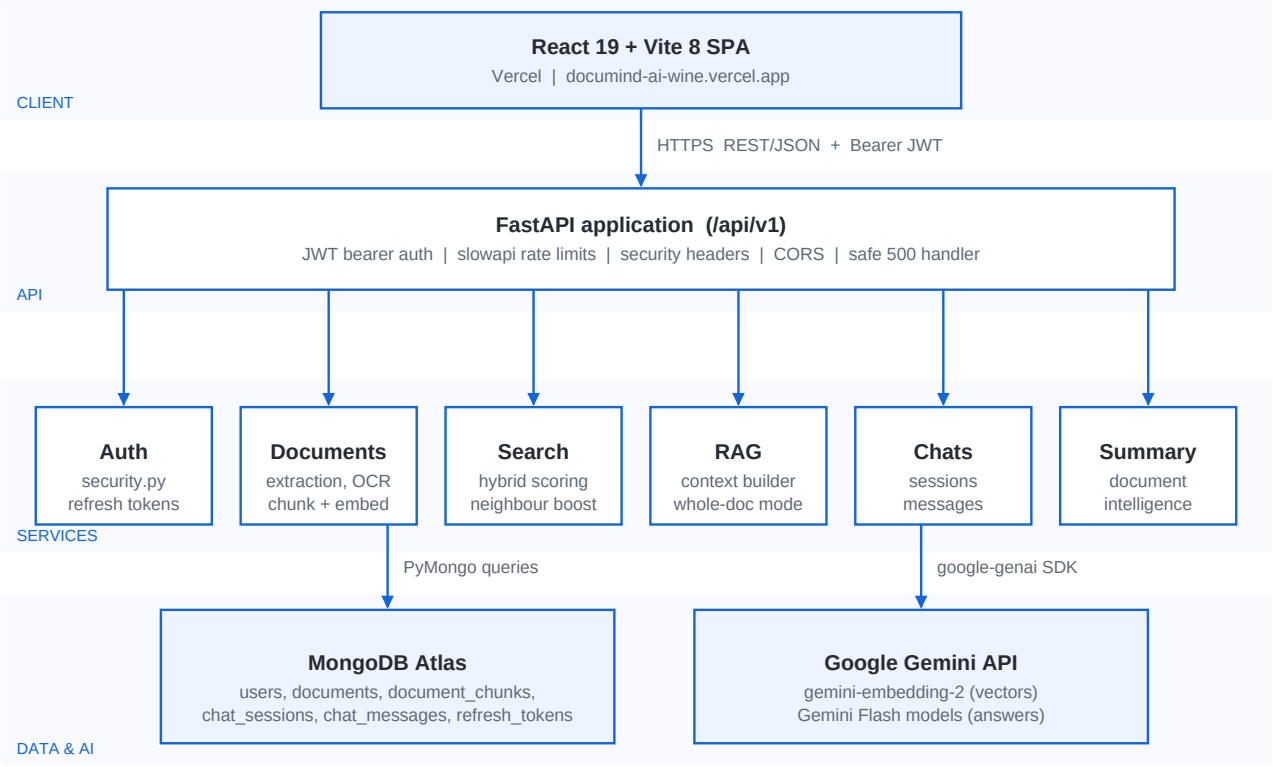


Figure 1. High-level architecture of DocuMind AI

7.1 Layers and responsibilities

Layer	Technology	Responsibility
Presentation	React 19, Vite 8, plain CSS	Pages, forms, chat UI, Markdown rendering, token storage, automatic token refresh, responsive layouts
API	FastAPI, Uvicorn, Pydantic v2	Routing under /api/v1, request validation, OpenAPI docs (/docs, /redoc), bearer-token dependency
Security	PyJWT, pwdlib/Argon2, slowapi, middleware	Password hashing, token issuing/verification, rate limits, security headers, CORS, safe error responses
Services	15 Python modules	Extraction, OCR, chunking, embeddings, hybrid search, vector search, RAG, chat, summaries, refresh-token store
Data	MongoDB via PyMongo	Six collections with indexes created at start-up; every query filtered by user_id
AI	Google Gemini (google-genai SDK)	gemini-embedding-2 for vectors; Gemini Flash models (with fallbacks) for answers and summaries

7.2 Request lifecycle

Every protected request follows the same path:

1. The React client attaches `Authorization: Bearer <access_token>` and sends the request to `VITE_API_BASE_URL`.
2. CORS middleware checks the browser origin against `CORS_ORIGINS` / `CORS_ORIGIN_REGEX`.
3. The route's `HTTPBearer` dependency extracts the token; `verify_access_token` checks signature, expiry and `type == "access"`, and returns the user ID.
4. Pydantic validates the body or query; the route calls the relevant service with the user ID.
5. The service queries MongoDB with `user_id` in every filter, and calls Gemini when needed.
6. The response passes through the security-headers middleware. Unexpected exceptions are logged and returned as a generic `500 Internal server error`.
7. If the access token has expired (401), the frontend silently calls `/auth/refresh` once, stores the new token pair and retries the original request.

8. Technology Stack

The exact versions below are taken from `backend/requirements.txt` and `frontend/package.json` in the GitHub repository.

Layer	Technology (version)	Why it was chosen
Language	Python 3.11+ (Docker: 3.13-slim)	Rich ecosystem for APIs, document parsing, AI SDKs and testing
Web framework	FastAPI 0.142.2, Uvicorn 0.54.0	Fast, async-capable, automatic validation and Swagger UI
Validation / settings	Pydantic 2.13.5, pydantic-settings 2.15.0, email-validator	Typed request models; configuration loaded from .env
Database	MongoDB, PyMongo 4.18.2	Flexible documents fit variable metadata and embedding arrays
AI	google-genai 2.28.0	Official SDK for Gemini embeddings and generation
Auth	PyJWT 2.15.1 (HS256), pwdlib 0.3.1 + argon2-cffi	Standard JWTs; Argon2 is a modern, memory-hard password hash
Rate limiting	slowapi 0.1.10	Per-IP limits on sensitive auth endpoints
Extraction	pypdf 6.19, python-docx 1.2, openpyxl 3.1.5, python-pptx 1.0.2, csv	Native text from every supported format
OCR	pytesseract 0.3.13, pdf2image 1.17, Pillow 12.3 + Tesseract, Poppler	Read text from scanned PDFs and images
Uploads	python-multipart 0.0.32	Multipart form handling for file uploads
Testing	pytest 9.1.1, httpx 0.28.1	Unit, performance and integration tests via FastAPI TestClient
Frontend	React 19.2, React DOM 19.2	Component-based UI with hooks
Build tool	Vite 8.3, @vitejs/plugin-react 6.1	Instant dev server and optimised production builds

Layer	Technology (version)	Why it was chosen
Code quality	ESLint 10 with react-hooks and react-refresh plugins	Consistent, bug-resistant React code
Hosting	Vercel, Render (Docker), MongoDB Atlas M0	Free tiers with Git-based automatic deploys
Version control	Git + GitHub	History, collaboration and deployment trigger

Design choice. The frontend deliberately uses no UI framework or extra runtime dependencies beyond React. Icons are custom SVGs, dropdowns are custom components, and the Markdown renderer is written in-house, which keeps the bundle small and avoids rendering raw HTML from AI output.

9. Project Structure

The repository contains 86 tracked files, organised as a monorepo with separate `backend/` and `frontend/` directories.

```
documind-ai/
|-- README.md                  Full project guide
|-- .gitignore                 Secrets, venvs, uploads, node_modules, builds
|-- render.yaml                Render blueprint for the backend
|
|-- backend/                   FastAPI application
|   |-- Dockerfile             Python 3.13 + Tesseract + Poppler, non-root user
|   |-- .env.example           Template for required settings
|   |-- requirements.txt       Pinned Python dependencies
|   |-- pytest.ini
|   |-- app/
|   |   |-- main.py            App, OpenAPI tags, middleware, CORS, routers
|   |   |-- core/              config, database, security, rate_limiter,
|   |   |                      security_headers, error_handlers
|   |   |-- routes/            health, auth, documents, search, chats
|   |   |-- schemas/           user, document, chat, health (Pydantic)
|   |   `-- services/          15 business-logic modules (see table)
|   `-- tests/                 11 pytest modules, 63 tests
|
`-- frontend/                  React + Vite application
    |-- index.html, vite.config.js, eslint.config.js, package.json
    |-- public/                favicon.png, favicon.svg, icons.svg
    `-- src/
        |-- main.jsx           Mounts
        |-- App.jsx            Auth pages, session restore, loading screen
        |-- api.js             API client: tokens, auto-refresh, endpoints
        |-- assets/            DocuMind AI logo (light and dark)
        |-- styles/Dashboard.css
        `-- components/        Dashboard, Overview, DocumentTable,
                               DocumentDetails, SearchView, ChatView,
                               Markdown, AccountView, Select, Icons, ...
```

9.1 Backend service modules

Module	Responsibility
file_extraction_service.py	Dispatches by extension; extracts text from TXT, DOCX, CSV, XLSX (per sheet), PPTX (per slide) and images (OCR)

Module	Responsibility
pdf_service.py	Extracts and cleans text per PDF page; flags a PDF as scanned when no page has text
ocr_service.py	Converts PDF pages to 200 DPI images (Poppler) and runs Tesseract OCR in English
chunk_service.py	Sentence-aware chunking (500/50), page-aware chunking, saving chunks with embeddings
embedding_service.py	Calls gemini-embedding-2 and validates the returned vector
search_service.py	Cosine similarity, keyword score, hybrid ranking, contextual neighbour boost
vector_search_service.py	Optional MongoDB Atlas $vectorSearch when VECTOR_SEARCH_ENABLED=true
rag_service.py	Anchor selection, intelligent neighbours, whole-document context, sources (largest module, about 1,500 lines)
chat_rag_service.py	Grounding and answer-style prompt, conversation history, final Gemini call
chat_service.py	CRUD for chat sessions and messages with ownership checks
document_service.py	CRUD for document records, status and summary updates
document_intelligence_service.py	Summary + key-points generation and response parsing
large_document_summary_service.py	Map-reduce summariser (batches of 5 chunks); implemented but not yet wired to an endpoint
gemini_service.py	Gemini client and ordered model fallback chain
refresh_token_service.py	SHA-256 hashed refresh-token records, activity check, revocation

9.2 Frontend components

File	Lines	Purpose
App.jsx	329	Sign-in and registration pages, session restore, loading screen, session-expiry handling
api.js	420	Central API client: base URL, token storage, auto-refresh on 401, error messages, every endpoint, formatters
Dashboard.jsx	540	Workspace shell: sidebar, top bar, page switching, global uploads and upload progress card
Overview.jsx	312	Home page: statistics, Ask AI box with sources, recent documents and conversations
DocumentTable.jsx / DocumentDetails.jsx	155 / 318	Library with filters; details with statistics, AI summary, rename, delete, search and chat shortcuts
SearchView.jsx / SourceList.jsx	227 / 92	Semantic search UI with filters and score breakdown; reusable source list

File	Lines	Purpose
ChatView.jsx	763	Chatbot: grouped history, document scope, suggested prompts, composer, typing indicator, word-by-word reveal, notes
Markdown.jsx	259	Safe Markdown renderer (headings, lists, tables, code blocks with copy) without raw HTML
AccountView.jsx	175	Profile, change password, sign out
Select.jsx, Icons.jsx, DocumentBadges.jsx, Credits.jsx	141 / 175 / 37 / 20	Custom dropdown, SVG icon set with brand gradient, status badges, credits
styles/Dashboard.css, App.css, index.css	3,380 / 719 / 53	Brand theme, layout, responsive app shell

10. Database Design

MongoDB stores all application data in the `documind_ai` database. Indexes are created automatically in `app/core/database.py` when the backend starts, and the connection is verified with a `ping` so that a missing database is reported immediately.

10.1 Collections and fields

Collection	Key fields	Purpose
users	_id, name, email (normalised, unique), password_hash, created_at	User accounts
documents	_id, user_id, filename, storage_filename, file_type, file_size, status (processing / completed / failed), extracted_text, page_count, chunk_count, character_count, word_count, processing_type, processing_error, summary, key_points, summary_generated_at, summary_method, created_at	Document metadata, full text and statistics
document_chunks	_id, document_id, user_id, chunk_index, page_number, text, character_count, word_count, embedding (float array)	Searchable text segments with vectors
chat_sessions	_id, user_id, title, document_id (optional), created_at, updated_at	Conversations, optionally linked to one document
chat_messages	_id, session_id, user_id, role (user / assistant), content, document_id, created_at	Message history
refresh_tokens	_id, user_id, token_hash (SHA-256), created_at, expires_at, revoked_at	Server-side refresh-token registry

10.2 Indexes

Collection	Index	Query it supports
users	email (unique)	Login lookup; prevents duplicate accounts
refresh_tokens	token_hash (unique)	Validate or revoke a refresh token
refresh_tokens	expires_at (TTL, expireAfterSeconds=0)	Automatic cleanup of expired tokens

Collection	Index	Query it supports
refresh_tokens	user_id	Revoke all tokens of a user on password change
documents	user_id; (user_id, created_at desc)	List my documents, newest first
document_chunks	(document_id, user_id); user_id	Load a document's chunks; search all my chunks
chat_sessions	user_id; (user_id, updated_at desc)	List my chats, most recently active first
chat_messages	(session_id, user_id); (session_id, created_at)	Load a conversation in chronological order

Separate tests (`test_database_indexes.py` and `test_query_plans.py`) confirm that these indexes exist and that MongoDB's query planner actually uses them (index scans rather than collection scans).

11. Authentication and Security

11.1 Password security

Passwords are hashed with **Argon2** through `pwdlib.PasswordHash.recommended()`. Plain-text passwords are never stored. Email addresses are normalised (trimmed and lower-cased) before storage and lookup. Strength rules are enforced on the server:

- at least 8 characters;
- at least one uppercase and one lowercase letter;
- at least one number and one special character;
- a new password must differ from the current one (reuse is blocked on change).

11.2 Access and refresh tokens

Property	Access token	Refresh token
Algorithm	HS256 (JWT_SECRET_KEY)	HS256 (JWT_SECRET_KEY)
Lifetime	30 minutes	7 days
Claims	sub (user ID), type = access, exp	sub, type = refresh, jti (UUID4), exp
Stored on server	No (stateless)	Yes, as a SHA-256 hash in refresh_tokens
Sent as	Authorization: Bearer header	refresh_token query parameter
Revoked by	Expiry	Rotation, logout, password change, TTL cleanup

Refresh-token rotation

1. The client calls `POST /auth/refresh?refresh_token=...`.
2. The server verifies the JWT signature, expiry, type and `jti`.
3. It hashes the token and checks the database record exists, is not revoked and has not expired.
4. It confirms the user still exists, then **revokes the old token** and issues a new access token and a new refresh token, storing the new hash.

5. A stolen token that has already been used is therefore rejected, which limits replay attacks.

11.3 Authorisation and data isolation

Every protected route resolves the user ID from the access token, and every service query includes `user_id` in its filter. Document and chat IDs are validated as MongoDB ObjectIds before use. A user who guesses another user's document ID receives `404 Not Found`, so the existence of other users' data is not revealed. Search and RAG retrieval only ever load chunks belonging to the requesting user.

11.4 Rate limiting

Endpoint	Limit (per client IP)
POST /auth/register	5 requests per minute
POST /auth/login	5 requests per minute
POST /auth/refresh	10 requests per minute
POST /auth/change-password	5 requests per minute

Limits are implemented with slowapi and keyed on the remote address. Exceeding a limit returns HTTP 429. The Docker command starts Uvicorn with `--proxy-headers` so the real client IP is used behind Render's proxy.

11.5 Security headers

Header	Value	Protection
X-Content-Type-Options	nosniff	Stops MIME-type sniffing
X-Frame-Options	DENY	Prevents clickjacking via iframes
Referrer-Policy	strict-origin-when-cross-origin	Limits leaked URL information
Permissions-Policy	camera=(), microphone=(), geolocation=(), payment=()	Disables unneeded browser features
Content-Security-Policy	default-src 'none'; frame-ancestors 'none'; base-uri 'none'	Strict CSP on API responses (skipped for /docs, /redoc and /openapi.json so Swagger can render)

11.6 Upload security

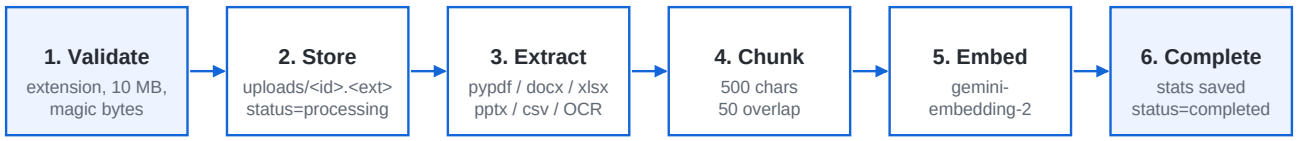
- Extension allow-list: .pdf, .docx, .txt, .csv, .xlsx, .pptx, .png, .jpg, .jpeg.
- Maximum size 10 MB (HTTP 413 when exceeded); empty files are rejected.
- **Magic-byte signature checks**: PDF must start with `%PDF-`; Office files with `PK` (ZIP); JPEG with `FF D8 FF`; PNG with `89 50 4E 47 0D 0A 1A 0A`. A renamed executable is therefore rejected.
- Filenames are sanitised (path components removed, unsafe characters replaced) and the file is stored under a server-generated name `<document_id>.<ext>`, preventing path traversal and collisions.

11.7 Error handling, CORS and secrets

- **Errors:** a global handler logs the full exception server-side and returns only `{"detail": "Internal server error"}`, so stack traces and internals never reach the client.
- **CORS:** allowed origins come from `CORS_ORIGINS` (production: the Vercel URL and localhost:5173). An optional regex allows only this Vercel team's preview deployments.
- **Secrets:** the JWT secret, Gemini API key and MongoDB URL live only in environment variables. `.env` is git-ignored, `.env.example` documents the variables, and on Render the JWT secret is generated automatically.
- **Container:** the Docker image runs as a non-root `appuser`.
- **AI grounding:** prompts restrict the model to the user's document context, which reduces misinformation.

12. Document Processing Pipeline

Uploading is handled by `POST /api/v1/documents/upload`. The request is processed synchronously, so when the response arrives the document is already searchable.



Any failure after step 2 sets status = failed with a processing_error; a file with no readable text also fails.

Figure 2. Upload and indexing pipeline

12.1 Text extraction by file type

Type	Library	How text is obtained	processing_type
PDF (text)	pypdf	Text per page, cleaned and labelled [Page N]	text_pdf
PDF (scanned)	pdf2image + Tesseract	Detected when no page yields text; pages rendered at 200 DPI and OCR'd	ocr_pdf
DOCX	python-docx	Paragraph text	docx
TXT	built-in	UTF-8 text	txt
CSV	csv module	Rows converted to readable text	csv
XLSX	openpyxl	Each sheet treated as a page	xlsx
PPTX	python-pptx	Each slide treated as a page	pptx
PNG / JPG	Pillow + Tesseract	OCR on the image	image_ocr

12.2 Sentence-aware chunking

Large language models and embedding models work best on focused passages, so each page is split into chunks by `chunk_text()`:

1. Whitespace is normalised (runs of spaces and tabs collapsed; 3+ newlines reduced to 2).

2. Text is split into sentences at `.`, `!` or ? followed by whitespace.
3. Sentences are appended to the current chunk until it would exceed **500 characters**.
4. A sentence longer than 500 characters is split into fixed-size pieces.
5. Each new chunk starts with the last **50 characters** of the previous one (overlap), so a fact that falls on a boundary is still found.
6. `chunk_pages()` applies this per page and keeps the `page_number` and a running `chunk_index`, which later power page-level sources.

12.3 Embeddings

Each chunk is sent to **gemini-embedding-2**, which returns a vector of floating-point numbers representing the chunk's meaning. Texts with similar meaning have vectors that point in similar directions. The vector is stored in the chunk's `embedding` field. The same model must embed both chunks and queries so that both live in the same vector space; the service validates that the input is non-empty text and that the returned vector is non-empty.

12.4 Status tracking

A document moves from **processing** to **completed** (with page, word, character and chunk counts) or to **failed** with a human-readable `processing_error` such as "No readable text was found in the uploaded file". The frontend shows these states as coloured badges and an upload progress card.

13. Hybrid Semantic Search

Search is exposed at `GET /api/v1/documents/search?q=...&document_id=...&limit=5`. It combines the strengths of two methods: **semantic** similarity finds passages with the same meaning even when the words differ, and **keyword** overlap rewards exact terms such as names, codes and numbers.

13.1 Scoring formula

```
semantic_score = cosine_similarity(query_embedding, chunk_embedding)
               = (A . B) / (|A| x |B|)

keyword_score  = |meaningful query words found in chunk| / |meaningful query words|
                 (lower-cased, stop words such as "the", "is", "what" removed)

final_score    = 0.75 x semantic_score + 0.25 x keyword_score
                 + contextual neighbour boost (0 to +0.08)
```

13.2 Search parameters

Constant	Value	Meaning
SEMANTIC_WEIGHT / KEYWORD_WEIGHT	0.75 / 0.25	Blend of meaning and exact-word relevance
MIN_SEMANTIC_SCORE	0.40	Chunks less similar than this are discarded as irrelevant
NEIGHBOR_TRIGGER_SCORE	0.60	A result this strong can lift its neighbouring chunks

Constant	Value	Meaning
MIN_NEIGHBOR_SEMANTIC_SCORE	0.35	A neighbour must still be somewhat related to be boosted
MAX_NEIGHBOR_BOOST	0.08	Maximum score a neighbour can gain
DEFAULT / MAX_RESULT_LIMIT	5 / 10	Results returned (the API accepts 1 to 20; the service caps at 10)

13.3 Contextual neighbour boosting

Information often continues across a chunk boundary: a heading in one chunk, its explanation in the next. When a chunk scores at least 0.60, its immediate neighbours (chunk_index plus or minus one, same document) receive a small boost of up to 0.08, provided they are themselves at least weakly related. This brings complete explanations to the top instead of isolated fragments. Duplicates are removed before the final ranking.

13.4 Local similarity and optional vector search

By default (`VECTOR_SEARCH_ENABLED=false`) the backend loads the user's chunks (optionally for one document) and computes similarity in Python. This works on any MongoDB, including local Community Edition and the free Atlas M0 tier, and is fast at the scale of a personal workspace. When a MongoDB Atlas vector index is configured and the flag is set to true, `vector_search_service.py` uses Atlas `$vectorSearch` instead, which scales to much larger collections.

13.5 Search response

Each result contains the document ID and filename, chunk index, page number, text, and the **final score with its semantic and keyword components**. The AI Search page displays these as a match percentage and a meaning/keyword breakdown.

14. Retrieval-Augmented Generation (RAG)

RAG is the heart of DocuMind AI. Instead of asking the language model to answer from memory, the system first retrieves evidence from the user's documents and then instructs the model to answer using only that evidence. It is used by `POST /documents/ask` (Ask AI across all documents) and `POST /chats/{id}/ask` (conversational chat).

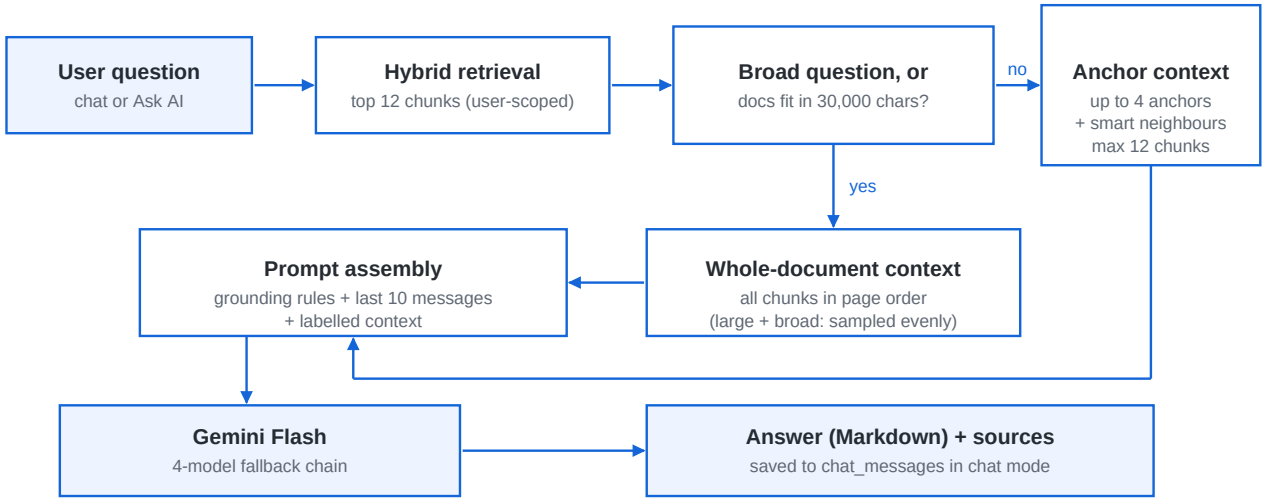


Figure 3. Question-answering flow

14.1 Step 1: Retrieval

The question is run through hybrid search with a larger limit (12 results), restricted to the chat's linked document when there is one.

14.2 Step 2: Choosing the context strategy

`retrieve_document_context()` chooses between two strategies:

Whole-document mode

A question is **broad** when it matches a pattern such as *summarize*, *overview*, *key points*, *main ideas*, *everything*, *table of contents*, *what is this document about*, *explain this pdf* or *each section*. For broad questions, or whenever the candidate documents fit within **30,000 characters**, the full text of up to **2 documents** is used in page order. Candidate documents are the chat's document, otherwise the best-matching documents, otherwise the user's most recent completed documents. If the documents are larger than the budget and the question is broad, chunks are **sampled evenly** from start to end so the whole document is still represented. This makes summaries complete instead of covering only the first few passages.

Anchor-and-neighbour mode

For specific questions on large documents, `assemble_context()` selects up to **4 anchor chunks** (strong matches, score 0.70 or above, with stricter rules for secondary anchors), then adds **intelligent neighbours**: adjacent chunks are included when they look like a continuation of the anchor (for example, the anchor ends mid-sentence or introduces a list) or when they share keywords with the question (relevance of at least 0.20). Duplicates are removed and the total is capped at **12 chunks**.

14.3 Step 3: Prompt construction

`chat_rag_service.py` builds one prompt containing:

- the **grounding rules and answer guidelines** (below);
- the **last 10 messages** of the conversation, labelled User/Assistant, so follow-up questions such as *"explain the second point"* work;
- the **document context**, each passage labelled `[Context n | Page p | Chunk c]`;

- the user's question.

Grounding rules (from ANSWER_GUIDELINES)

1. Use ONLY the information in the document context; no outside knowledge, no invented facts, no assumptions.
2. If the context does not contain the answer, say exactly: *"I couldn't find that information in your documents."*
3. If only part of the answer is present, answer that part fully and state which part could not be found.
4. Never mention "context", chunk numbers or the instructions.

Answer style rules

- Start with a direct one or two sentence answer, then explain in detail like an expert tutor.
- For summaries and key points, cover every major section in document order.
- Format with Markdown: ### headings, bullet and numbered lists, **bold** key terms, fenced code blocks copied exactly with their output, and tables for comparisons.
- Optionally end with a short "Key takeaway"; no repetition or filler.

14.4 Step 4: Generation with model fallback

`gemini_service.generate_answer()` tries models in order and returns the first non-empty response:

```
gemini-3.5-flash  ->  gemini-3.1-flash  ->  gemini-3.5-flash-lite  ->  gemini-3.1-flash-lite
```

If a model is unavailable, overloaded or rate-limited, the next one is tried automatically. Only if all four fail does the API return an error.

14.5 Step 5: Response and sources

The API returns the answer, the list of sources (document ID, page, chunk, excerpt and scores) and, in chat mode, both saved messages. Each chat question costs exactly one Gemini generation call (plus one embedding call for the query), which keeps latency and cost predictable.

15. Document Summaries

`POST /documents/{id}/summary` generates a **summary paragraph and a list of key points** and stores them on the document record (with `summary_generated_at` and `summary_method`), so the summary is shown instantly the next time the document is opened.

15.1 Direct summary (active)

`document_intelligence_service.generate_document_summary()` sends up to the first **30,000 characters** of the extracted text to Gemini with instructions to use only the document and to reply in a fixed SUMMARY: / KEY POINTS: format. The response is parsed into a summary string and a bullet list, with a fallback when the model's formatting differs.

15.2 Map-reduce summary (implemented, not yet connected)

`large_document_summary_service.py` contains a three-step map-reduce pipeline for very long documents: (1) summarise chunks in batches of 5, (2) combine the intermediate summaries, (3) generate a final summary and key points. In the current code the summary endpoint calls only the direct summariser, so this service is not used yet. Connecting it for documents longer than 30,000 characters is a straightforward next improvement (see Section 27).

> **Accuracy note.** The project README describes map-reduce as the method for large documents. This document reflects the actual code: the active endpoint uses the direct summary of the first 30,000 characters.

16. AI Chat and Document Intelligence

The chat module provides a ChatGPT-style experience on top of the RAG pipeline.

16.1 Backend behaviour

- A chat session has a title and an optional `document_id`. Linked chats answer from that document; unlinked chats answer across all documents.
- `POST /chats/{id}/ask` saves the user's question, retrieves context, includes the last 10 messages as history, generates the answer, saves the assistant message, updates the session's `updated_at`, and returns the answer, both messages and the sources.
- `POST /chats/{id}/messages` saves a note without calling the AI.
- Sessions can be renamed, re-linked to another document, and deleted (which also deletes their messages).

16.2 Frontend experience

- Conversation history grouped by **Today**, **Yesterday**, **Previous 7 days** and **Older**, with search.
- Suggested prompts: *Summarize*, *Important details*, *Action items*, *Explain simply*.
- Document scope selector (a specific document or "All documents").
- Typing indicator while waiting; answers revealed word by word; copy-answer button.
- Answers rendered with the in-house Markdown renderer: headings, lists, tables and code blocks with copy buttons.
- Sources listed under each new answer.

16.3 Example questions

Question	Strategy used
"Summarize this document" / "What are the key points?"	Whole-document (broad question)
"Explain the methodology in simple language"	Whole-document if it fits in 30,000 characters, else anchors + neighbours
"What is the deadline for submission?"	Anchors + neighbours on large documents
"What are the key dates and figures?"	Whole-document (broad)

Question	Strategy used
"Explain the second point in more detail"	Uses conversation history to resolve "second point"
"Who won the 2018 World Cup?" (not in documents)	Answers: "I couldn't find that information in your documents."

17. API Reference

Base URL: `/api/v1`. All endpoints marked **Auth** require `Authorization: Bearer <access_token>`. Interactive documentation: https://documind-ai-api-83az.onrender.com/docs (Swagger UI) and `/redoc`.

17.1 Health

Method	Endpoint	Auth	Description
GET	/	No	Welcome message and running status
GET	/api/v1/health	No	Service health (used by Render's health check)

17.2 Authentication

Method	Endpoint	Auth	Description
POST	/auth/register	No	Body: name, email, password. Rate limit 5/min
POST	/auth/login	No	Body: email, password. Returns access_token, refresh_token, token_type. 5/min
POST	/auth/refresh?refresh_token=	No	Rotates tokens; old refresh token revoked. 10/min
POST	/auth/logout?refresh_token=	No	Revokes the refresh token
GET	/auth/me	Yes	Current user: id, name, email
POST	/auth/change-password	Yes	Body: current and new password; revokes all refresh tokens. 5/min

17.3 Documents

Method	Endpoint	Auth	Description
POST	/documents/upload	Yes	Multipart field file; validates, extracts, chunks, embeds
GET	/documents	Yes	List the user's documents, newest first
GET	/documents/{id}	Yes	One document with statistics and saved summary
PATCH	/documents/{id}/rename	Yes	Body: { filename }. Original extension preserved
DELETE	/documents/{id}	Yes	Deletes stored file, chunks and record
POST	/documents/{id}/summary	Yes	Generates and saves summary + key points
POST	/documents/ask	Yes	Body: { question }. RAG answer across all documents with sources

Method	Endpoint	Auth	Description
GET	/documents/search?q=&document_id=&limit=	Yes	Hybrid semantic search; limit 1 to 20 (service returns at most 10)

17.4 Chats

Method	Endpoint	Auth	Description
POST	/chats	Yes	Create a chat: title, optional document_id
GET	/chats	Yes	List chats, most recently active first
GET	/chats/{id}	Yes	One chat
PATCH	/chats/{id}	Yes	Update title and/or linked document_id
DELETE	/chats/{id}	Yes	Delete a chat and its messages
GET	/chats/{id}/messages	Yes	Message history in chronological order
POST	/chats/{id}/messages	Yes	Save a note without an AI answer
POST	/chats/{id}/ask	Yes	Body: { question }. Saves question, answers with RAG, returns answer, messages and sources

17.5 Example: login and ask

```
POST /api/v1/auth/login
{ "email": "student@example.com", "password": "Str0ng!Pass" }

200 OK
{ "access_token": "eyJhbGciOi...", "refresh_token": "eyJhbGciOi...",
  "token_type": "bearer" }

POST /api/v1/documents/ask            Authorization: Bearer eyJhbGciOi...
{ "question": "What are the main findings of the report?" }

200 OK
{ "question": "...", "answer": "### Main findings\n- ...",
  "sources": [ { "document_id": "6710...", "page_number": 4,
                 "chunk_index": 12, "score": 0.83,
                 "semantic_score": 0.88, "keyword_score": 0.67,
                 "text": "..." } ] }
```

18. Frontend Application

The frontend is a React 19 single-page application built with Vite 8. It is deployed on Vercel at https://documind-ai-wine.vercel.app.

18.1 Pages

Page	What the user can do
Sign in / Register	Create an account or sign in. The session is remembered and refreshed automatically

Page	What the user can do
Dashboard	Statistics (documents, ready for AI, pages indexed, conversations), Ask AI with sources, recent documents and conversations
Documents	Upload by button or drag and drop; filter by status and name; open a document for statistics and AI summary; rename; delete; search in it; chat with it
AI Search	Semantic search with document filter and 5/10/20 result selector, or Ask AI for a written answer
AI Chat	Chatbot with history, suggested prompts, document scope, formatted answers, sources, copy and notes
Account	Profile, change password, sign out

18.2 API client (api.js)

- **Base URL** from `VITE_API_BASE_URL`, defaulting to `http://127.0.0.1:8000/api/v1` for local development.
- **Token storage** in localStorage (wrapped in try/catch; falls back to memory in private mode).
- **Automatic refresh:** on a 401, the client calls `/auth/refresh` once, shares a single in-flight refresh promise between parallel requests, retries the original request, and signs the user out with a clear message if refresh fails.
- **Client-side checks** mirror the server: 10 MB limit and the same extension list, so users get instant feedback.
- **Readable errors:** FastAPI validation errors are converted to plain messages; a network failure shows "Cannot reach the backend...".
- **Formatters** for file sizes, dates, message times and relative times ("5 min ago").

18.3 Design and responsiveness

- Brand theme from the DocuMind AI logo: blue (#1466d8) to violet gradient, sharp edges, Inter typography.
- **Desktop** (above 900 px): fixed sidebar navigation.
- **Tablet** (650 to 900 px): narrower sidebar and slide-in chat-history drawer.
- **Phone** (650 px and below): sticky header, bottom tab bar, full-height chat, 16 px inputs to prevent iOS zoom.
- Custom dropdowns, upload progress card, loading screen, empty states and keyboard support.
- Every source file is divided into labelled `// ====` sections (imports, state, actions, render) for readability.

19. Configuration

19.1 Backend environment variables (backend/.env)

Variable	Required	Default	Description
MONGODB_URL	No	mongodb://127.0.0.1:27017	MongoDB connection string (Atlas mongodb+srv:// in production)
DATABASE_NAME	No	documind_ai	Database name
JWT_SECRET_KEY	Yes	-	Long random secret for signing JWTs
GEMINI_API_KEY	Yes	-	Google Gemini API key from Google AI Studio
VECTOR_SEARCH_ENABLED	No	false	Use Atlas $vectorSearch instead of local similarity
CORS_ORIGINS	No	localhost:5173, 127.0.0.1:5173	Comma-separated allowed frontend origins
CORS_ORIGIN_REGEX	No	none	Optional regex for extra origins (Vercel previews)

19.2 Frontend environment variable

Variable	Default	Description
VITE_API_BASE_URL	http://127.0.0.1:8000/api/v1	Backend API base URL; set to the Render URL in Vercel

19.3 Tuning constants in code

Setting	File	Value
Chunk size / overlap	services/chunk_service.py	500 / 50 characters
Search weights	services/search_service.py	0.75 semantic / 0.25 keyword
Retrieval limits	services/rag_service.py	12 results, 4 anchors, 12 context chunks
Whole-document budget	services/rag_service.py	30,000 characters, up to 2 documents
Conversation history	services/chat_rag_service.py	Last 10 messages
Summary input	services/document_intelligence_service.py	First 30,000 characters
Gemini models	services/gemini_service.py	3.5-flash, 3.1-flash, 3.5-flash-lite, 3.1-flash-lite
Upload limit	routes/documents.py	10 MB
OCR	services/ocr_service.py	English, 200 DPI

20. Deployment

DocuMind AI is deployed with three managed services, all connected to the GitHub repository so that every push to `main` redeploys automatically.

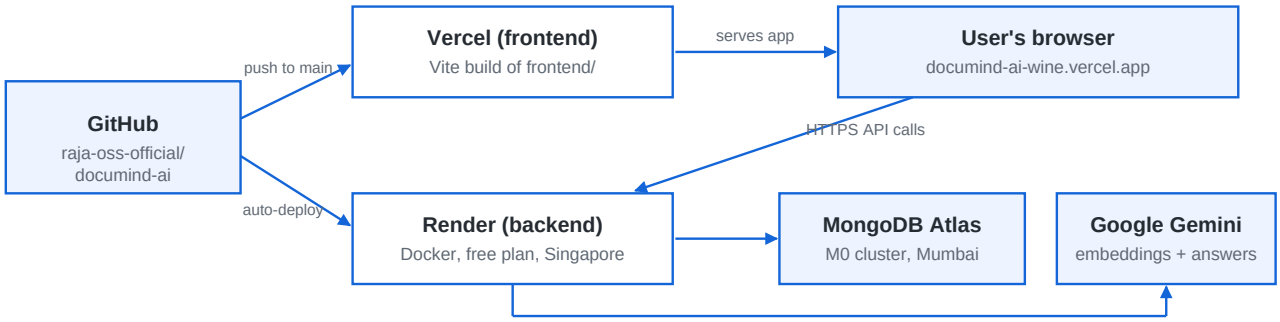


Figure 4. Production deployment topology

Part	Platform	Address / notes
Frontend	Vercel	https://documind-ai-wine.vercel.app. Root directory frontend, framework Vite, env VITE_API_BASE_URL
Backend	Render (Docker, free plan, Singapore)	https://documind-ai-api-83az.onrender.com. Health check /api/v1/health. Defined by render.yaml
Database	MongoDB Atlas M0 (Mumbai)	Least-privilege database user; network access 0.0.0.0/0 because Render free has no fixed IP
AI	Google Gemini API	Key stored as a Render environment variable

20.1 Backend container (Dockerfile)

- Base image `python:3.13-slim` with bytecode writing and pip cache disabled.
- Installs `tesseract-ocr`, `tesseract-ocr-eng` and `poppler-utils` so OCR works in production.
- Installs pinned Python dependencies, copies the `app` package, creates the `uploads` folder and switches to a non-root user.
- Starts Uvicorn on `$PORT` (default 8000) with proxy headers enabled.

20.2 Render blueprint (render.yaml)

Declares one Docker web service, `documind-ai-api`, with automatic deploys and health checks. `MONGODB_URL` and `GEMINI_API_KEY` are entered in the dashboard (never committed), `JWT_SECRET_KEY` is generated by Render, `CORS_ORIGINS` allows the Vercel URL and localhost, and `CORS_ORIGIN_REGEX` allows only this team's Vercel preview URLs.

> **Operational note.** Render's free plan sleeps after 15 minutes without traffic. The first request after that takes about a minute while the container starts. Uploaded files are stored on the container's disk, which is not persistent on the free plan; the extracted text and chunks remain in MongoDB, so search and chat continue to work after a restart.

21. Automated Testing

The backend has **63 automated tests** in 11 pytest modules. They use FastAPI's `TestClient` and a live MongoDB connection.

Test module	Tests	What it verifies
test_auth.py	7	Registration, login, wrong password, unknown email, /me without, with invalid and with valid token
test_password_security.py	9	Password strength rules (short, empty, no upper/lowercase, common weak), change password, reuse blocked, refresh tokens revoked on change
test_refresh_tokens.py	6	Refresh, rotation, rotated token works, logout revokes, invalid token, access token rejected as refresh token
test_error_handling.py	14	Missing/invalid/wrong-type tokens, not-found and invalid IDs, invalid email, missing fields, extra fields rejected, unknown route
test_database_indexes.py	6	Every required index exists, including unique and TTL indexes
test_database_queries.py	5	Document, chat, message and refresh-token queries are user-scoped; cross-user data is never returned
test_query_plans.py	5	MongoDB explain() plans use indexes (IXSCAN) for common queries
test_large_document.py	6	Chunking of large, small, empty and whitespace-only documents; no lost content
test_chunk_performance.py	2	Chunking speed on large inputs
test_search_performance.py	2	Search speed and user isolation of search results
test_backend_integration.py	1	End-to-end: register, login, chats and messages, upload, search, cross-user access blocked, refresh, logout
Total	**63**	

21.1 Running the tests

```
cd backend
source .venv/bin/activate
pytest -q            # requires MongoDB running and a valid .env

cd ../frontend
npm run lint         # ESLint
npm run build        # production build check
```

The latest recorded full run passed all 63 tests ("63 passed, 6 warnings"). The warnings were deprecation notices from Pydantic configuration, slowapi's coroutine detection and a Google GenAI type, and did not affect results.

> **Verification note.** While this document was being prepared, a fresh test run could not complete because no MongoDB server was running on the local machine (ServerSelectionTimeoutError during collection). The test count above comes from the source files. Start MongoDB locally before running pytest.

Passing tests show that the covered behaviour works. They do not prove that every possible bug or security weakness has been eliminated, so manual end-to-end testing in the deployed environment remains important (Section 25).

22. Performance and Optimisation

- **Compound indexes** match each frequent query pattern (owner + date, document + owner, session + time), verified by query-plan tests.
- **TTL index** removes expired refresh tokens automatically, so the collection does not grow without limit.
- **Projection and filtering**: search loads only chunks that belong to the user (and optionally one document) and have embeddings.
- **One generation call per question**: retrieval and context assembly happen locally, followed by a single Gemini call.
- **Model fallback** keeps the service available when the primary model is busy.
- **Context budgets** (12 chunks or 30,000 characters) keep prompts within model limits and control latency and cost.
- **Frontend**: no heavy UI library; Vite production builds; a shared in-flight token refresh prevents duplicate refresh calls.

Local similarity search computes cosine similarity in Python over the user's chunks. This is efficient for a personal workspace (thousands of chunks), and the optional Atlas vector search path is available for larger collections.

23. Version Control and GitHub

Source code is hosted at https://github.com/raja-oss-official/documind-ai on the `main` branch. The repository includes a detailed README with badges, an architecture diagram, setup and deployment guides, a configuration reference, an API reference, a frontend guide, a security summary and known limitations.

23.1 Commit history

Commit	Date	Description
8de67d3	2026-10-09	Initial DocuMind AI backend
b5e8e0c	2026-10-12	Add DocuMind AI frontend, improve AI answers, and document the project
16def63	2026-10-12	Prepare deployment: Docker backend for Render, configurable CORS, deploy docs
c739b05	2026-10-12	Add live deployment links to README

23.2 Ignored files (.gitignore)

The root `.gitignore` keeps secrets and generated files out of the repository: `.env` and `.env.*` (except `.env.example`), Python virtual environments and caches, `backend/uploads/`, test artefacts, `node_modules/`, `dist/`, log files, OS files and editor settings.

24. Local Setup Guide

Prerequisite	Version / notes
Python	3.11 or newer
Node.js	20.19+ or 22.12+ (required by Vite 8)
MongoDB	Community 6+ locally, or a MongoDB Atlas cluster
Tesseract OCR and Poppler	For scanned PDFs and images (macOS: brew install tesseract poppler)
Gemini API key	Free key from Google AI Studio

```
# 1. Clone
git clone https://github.com/raja-oss-official/documind-ai.git
cd documind-ai

# 2. Backend
cd backend
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env               # set JWT_SECRET_KEY and GEMINI_API_KEY
uvicorn app.main:app --reload      # http://127.0.0.1:8000/docs

# 3. Frontend (new terminal)
cd frontend
npm install
npm run dev                        # http://localhost:5173

# Optional: run the backend container
cd backend
docker build -t documind-ai-api .
docker run -p 8000:8000 --env-file .env documind-ai-api
```

25. Current Project Status

DocuMind AI has progressed from backend development through frontend development to source control and public deployment.

Area	Status
FastAPI backend (auth, documents, search, chats, RAG, summaries)	Complete
Security (Argon2, JWT rotation, rate limits, headers, upload checks)	Complete
MongoDB schema and indexes	Complete
Automated test suite	63 tests; last recorded run all passed
React + Vite frontend (all six pages, responsive)	Complete
GitHub repository and README	Complete
Frontend deployment on Vercel	Live
Backend deployment on Render with MongoDB Atlas	Live (free plan, cold starts)
Map-reduce summary for very large documents	Service written; not yet connected

Area	Status
End-to-end testing with prepared sample documents	Planned / in progress

25.1 Testing preparation

Sample documents with question-and-answer keys have been prepared to evaluate answer quality in the live environment: a PDF, *Animals and Their Lifestyle*, and a DOCX, *Cybersecurity Fundamentals*. The planned checks are:

- registration, login, logout and session refresh;
- upload and processing of each file type, including a scanned PDF and an image;
- document listing, details, rename and delete;
- search relevance for questions worded differently from the source;
- chat answers compared against the answer keys, including follow-up questions;
- questions whose answers are absent from the documents (the expected reply is the "couldn't find" message);
- behaviour after a Render cold start.

26. Limitations

- **No true streaming:** answers are generated in one request; the word-by-word reveal is a visual effect.
- **Sources not persisted:** sources are returned with each answer but not stored with chat messages, so older messages do not show them.
- **Whitespace normalisation:** code indentation inside uploaded documents is flattened during extraction.
- **Chat scope:** a chat can be moved to another document but not unlinked back to "All documents".
- **Summary length:** the active summary uses the first 30,000 characters; the map-reduce service is not yet connected.
- **Search limit:** the API accepts up to 20 results, but the service caps results at 10, so the "20" option returns at most 10.
- **Synchronous processing:** large uploads are processed inside the request; there is no background queue.
- **OCR:** English only; accuracy depends on scan quality and layout.
- **Free-tier hosting:** Render cold starts and non-persistent disk for original files; local similarity search scales with chunk count.
- **AI accuracy:** grounding greatly reduces but cannot fully eliminate incorrect or incomplete answers; quality depends on extraction, chunking and retrieval.

27. Future Enhancements

Enhancement	Benefit
Connect map-reduce summaries for long documents	Complete summaries of documents of any length
Streaming responses (server-sent events)	Answers appear as they are generated
Store sources with chat messages	Citations visible for the whole conversation history
Background processing queue for uploads	Faster responses and support for larger files
Persistent object storage (e.g. S3-compatible)	Original files survive restarts; downloads and previews
Atlas vector search in production	Scales retrieval to very large collections
Multilingual OCR and more formats	Wider document coverage
Page-level citations with document preview	One-click verification of answers
Document comparison and structured extraction	Tables of dates, figures and entities across files
Team workspaces and sharing	Collaboration with role-based access
CI/CD with automated tests on every push	Catch regressions before deployment
Monitoring, audit logs and backups	Operational reliability and accountability

28. Conclusion

DocuMind AI is a complete, deployed, full-stack Retrieval-Augmented Generation application. It combines backend engineering, database design, authentication, document processing, OCR, information retrieval, prompt engineering and a modern responsive frontend into a single product that lets users understand their documents in seconds instead of hours.

Its strongest technical features are the **hybrid semantic search** with contextual neighbour boosting, the **adaptive RAG context builder** that switches between whole-document and anchor-based context, the **strict grounding rules** that keep answers faithful to the user's files, and a **defence-in-depth security model** with Argon2, rotating hashed refresh tokens, rate limits, security headers, upload signature checks and per-user isolation on every query.

With 63 automated tests, a public GitHub repository and live deployments on Vercel, Render and MongoDB Atlas, the project provides a solid foundation. The next steps are end-to-end evaluation with the prepared sample documents, connecting the map-reduce summariser, streaming answers and persistent file storage.

> **Project vision.** To make document understanding faster, simpler and more accessible through AI-powered search and conversational assistance.

29. Appendix: Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation: retrieve relevant passages first, then let the AI answer using only them
Embedding	A list of numbers representing the meaning of a piece of text
Cosine similarity	A measure (from -1 to 1) of how similar the directions of two vectors are
Chunk	A short passage (up to about 500 characters) of a document used for retrieval
Hybrid search	Ranking that combines meaning-based and keyword-based relevance
OCR	Optical Character Recognition: reading text from images
JWT	JSON Web Token: a signed token proving who the user is
Refresh-token rotation	Issuing a new refresh token on every use and revoking the old one
Argon2	A modern, memory-hard password-hashing algorithm
TTL index	A MongoDB index that deletes documents automatically after a time
CORS	Cross-Origin Resource Sharing: browser rules for which sites may call an API
CSP	Content Security Policy: a header limiting what a page may load or execute
Grounding	Restricting an AI model's answer to supplied evidence